ConvPE: Convolutional Position Embedding Improves Transformers

Wang Zehao
Nanjing University
wzhagi@163.com

ABSTRACT

ConvPE (Convolutional Position Embedding) substantially improves both the performance and sample efficiency of Transformer models. At the cost of only 2.64% more parameters and a 2.70% increase in forward-pass time, it achieves a 2.82% reduction in training loss and substantial improvements on most evaluation benchmarks. It also requires only 68.7% of the training tokens needed by the baseline to reach the baseline's final training loss. ConvPE uses convolution to generate attention-score biases, making it compatible with various Transformer architectures and naturally extendable to high-dimensional modalities. The code is publicly available at `https://github.com/wzhagi/convpe`.

Keywords Position Embedding · Language Models · Transformers · Attention · Vision Transformers

1 Introduction

Transformers [1] are insensitive to input order. However, word order is crucial for language understanding. Explicit or implicit position embeddings are therefore necessary to inject order information into the model [2,3,4].

This paper proposes ConvPE, a novel and efficient position embedding scheme. ConvPE embeds position information into attention score biases, trading a minor parameter overhead for significant performance gains, as shown in Figure 1. Its design allows the model to switch freely between different attention masks and is compatible with various Transformer architectures. It also naturally supports high-dimensional modalities with n-dimensional convolution.

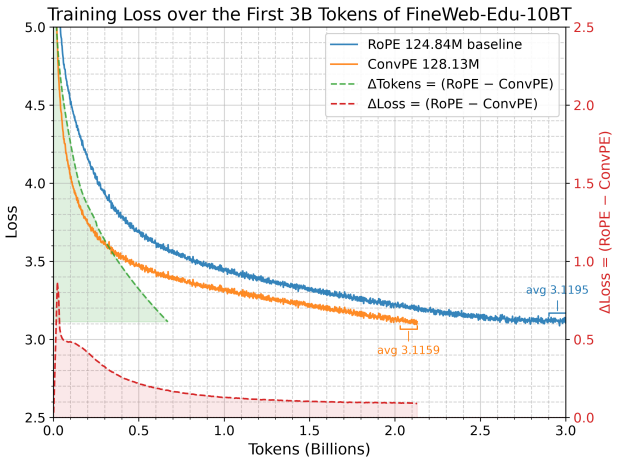


Figure 1

The main contributions of this paper are as follows:

1. We propose the ConvPE architecture that combines high performance and broad compatibility.

2. We study the influence of ConvPE hyperparameters and architecture.

3. We verify the effectiveness of ConvPE in language model pretraining.

4. We verify the effectiveness of ConvPE2D in visual model training.

5. We developed high performance TileLang [5] kernels for ConvPE.

2 Background

2.1 Attention Mechanism

Given the input query $Q \in \mathbb{R}^{S_q \times D_k}$, key $K \in \mathbb{R}^{S_{kv} \times D_k}$, and value $V \in \mathbb{R}^{S_{kv} \times D_v}$, the computation of scaled dot-product attention [1] $O \in \mathbb{R}^{T \times D_v}$ is:

$$O = \text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + \mathbf{B}\right) V$$

T is the query sequence length. S is the key/value sequence length. $\mathbf{B} \in \mathbb{R}^{S_q \times S_{kv}}$ is the attention score bias. Causal models introduce a causal mask $M \in \mathbb{R}^{S_q \times S_{kv}}$ that sets the attention score bias $\mathbf{B}$ for invisible positions to $-\infty$. With batch size and multi-head settings, Q, K, V, O additionally have B, H dimensions.

2.2 Position Embedding

Existing position embedding schemes can be roughly divided into five categories:

1. **Absolute Position Embedding** [1]: adds a fixed positional bias, usually sine waves, to the input vector. It is simple to implement, but its performance is usually limited.

2. **Learnable Positional Bias** [2,3]: adds a learnable positional bias to the input sequence. It is typically adopted in limited-window settings, such as spatial dimensions in vision models or fixed-rate compression.

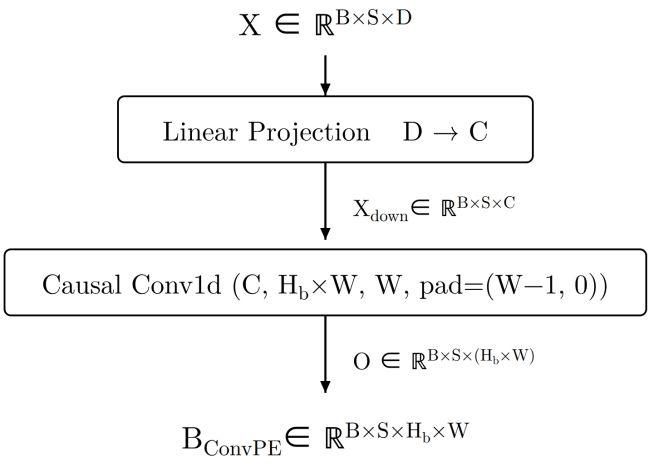
3. **Rotary Position Embedding** [4]: encodes position as a rotation matrix and applies it to queries and keys, so that attention scores depend on relative positions. RoPE has been widely adopted in modern large language models, enabling long-context extension via frequency scaling, such as YaRN [6].

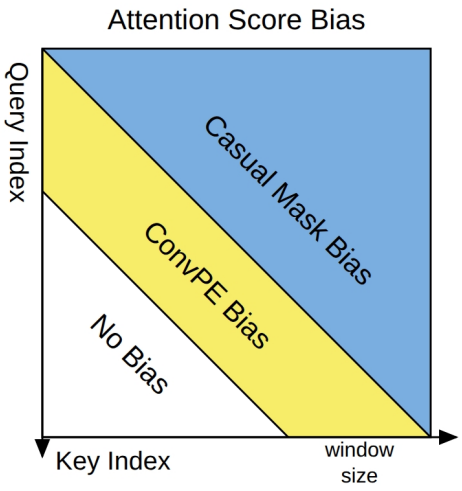
4. **Attention Score Bias**: adds a positional bias B to attention scores [7 8 9]. RoPE can be viewed as a rotation-based equivalent, as it also makes attention scores depend on relative positions.

5. **No Position Embedding (NoPE)** [10]: adds no explicit position embeddings; instead, it relies on model structure and causal attention masks to implicitly learn positional information requiring no explicit Position Embedding [11].

3 Method

ConvPE use a convolution to generates windowed attention score bias $\mathbf{B}_{ConvPE} \in \mathbb{R}^{B \times S \times H_b \times W}$ for self-attention tokens $X \in \mathbb{R}^{B \times S \times D}$ within the convolution window.





$$\mathbf{B}^{b,h,s_q,s_{kv}} = \begin{cases} \mathbf{B}^{b,s_q,h\%H_b,w}_{\text{ConvPE}}, & w \in [0, W-1] \text{ where } w = s_{kv} - s_q + W - 1 \\ 0, & \text{otherwise.} \end{cases}$$

The hyperparameters are:

- W: convolution window size.
- C: out features of linear projection and in channels of convolution.
- H_b: number of score bias heads.

The convolution takes C input channels, uses a kernel of size W, and outputs $H_b \times W$ channels. The projection has $D \times C$ parameters and the convolution has $W^2 \times C \times H_b$ parameters; the computation (MACs) per token is the same as the parameter count.

To reduce the convolution parameter count, we introduce three additional hyperparameters:

- Groups: number of convolution groups;
- Share: whether to share weights across convolution groups;
- Rank: rank used in the low-rank convolution.

4 Experiments

All models in our experiments use Multi-Query Attention (MQA)[12], with $K = V$[13]. Each attention layer has $D_{model} \times D_{head} \times (2 \times N_{heads} + 1)$ parameters. Each feed-forward layer uses SwiGLU[14] and has $8 \times D_{model}^2$ parameters.

4.1 Hyperparameter Search

We train each model for one epoch on WikiText-103[15] and record the validation loss. Training uses the Hugging Face Trainer[16], AdamW[17], and the GPT-2 tokenizer[3]. The training hyperparameters are: batch size 4, sequence length 1024, learning rate 3×10^{-4} with cosine learning-rate scheduler, and seed 42.

We train two types of backbone models, a tiny backbone with $N_{layers} = 12$, $D_{model} = 192$, $N_{heads} = 3$, $D_{head} = 64$, and a base backbone with $N_{layers} = 12$, $D_{model} = 768$, $N_{heads} = 12$, $D_{head} = 128$. The model name format is "`backbone RoPE/NoPE` W C Rank H_b Groups Share". For example, "`tiny rope w128d6r128h3g1s`" denotes a tiny backbone using RoPE with $W = 128$, $C = 6$, Rank $= 128$, $H_b = 3$, Groups $= 1$, and Share weights. For simplicity, model names may omit the default configuration: `NoPE`, `r0`(no low-rank convolution), `h1` ($H_b = 1$), `g1`(Groups $= 1$), and `s0` (no weight sharing).

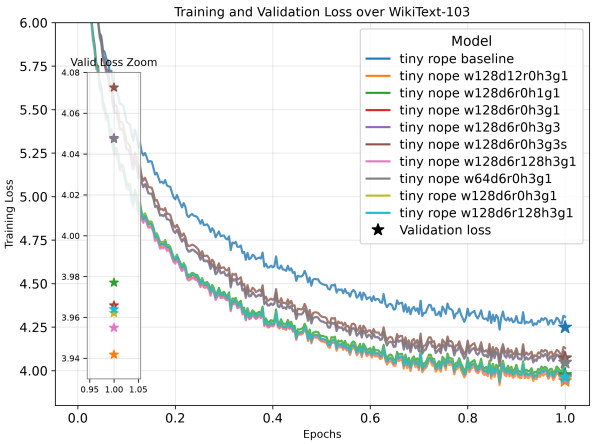


Figure 2: Tiny model training curve on WikiText-103

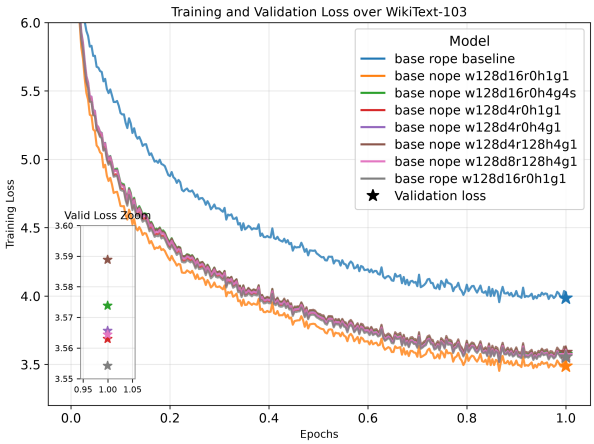


Figure 3: Base model training curve on WikiText-103

First, we train tiny models with hyperparameters around "`nope w128d6r0h3g1`". Figure 2 shows that all ConvPE variants outperform the RoPE baseline. The largest gains come from increasing the window size and the projection dimension. Using grouped convolution causes the largest performance drop. RoPE integration and other parameter-saving methods show minor differences from "`nope w128d6r0h3g1`".

We then train base model with hyperparameters around "`nope w128d4r0h4g1`" and "`nope w128d16r0h1g1`". Figure 3 shows that $H_b = 1$ without any parameter-saving technique achieves the best balance between performance and parameter efficiency.

In additional experiments, we also try using X , Q[9] and KV as the convolution input, but performance drops.

4.2 Pretraining

We choose "`base w128d16`" as the ConvPE variant configuration for performance and throughput considerations. We train the RoPE baseline `base RoPE` and the ConvPE variant from scratch on the first 3B tokens of FineWeb-Edu-10BT. We use Muon[18] to optimize the 2D parameters, excluding the embedding layer and the language head, with a learning rate of 3×10^{-3}, while Adam[19] optimizes the remaining parameters at a learning rate of 3×10^{-4}, both using a cosine learning-rate scheduler. The training hyperparameters are: batch size 4, gradient accumulation steps 4, sequence length 1024, and seed 42.

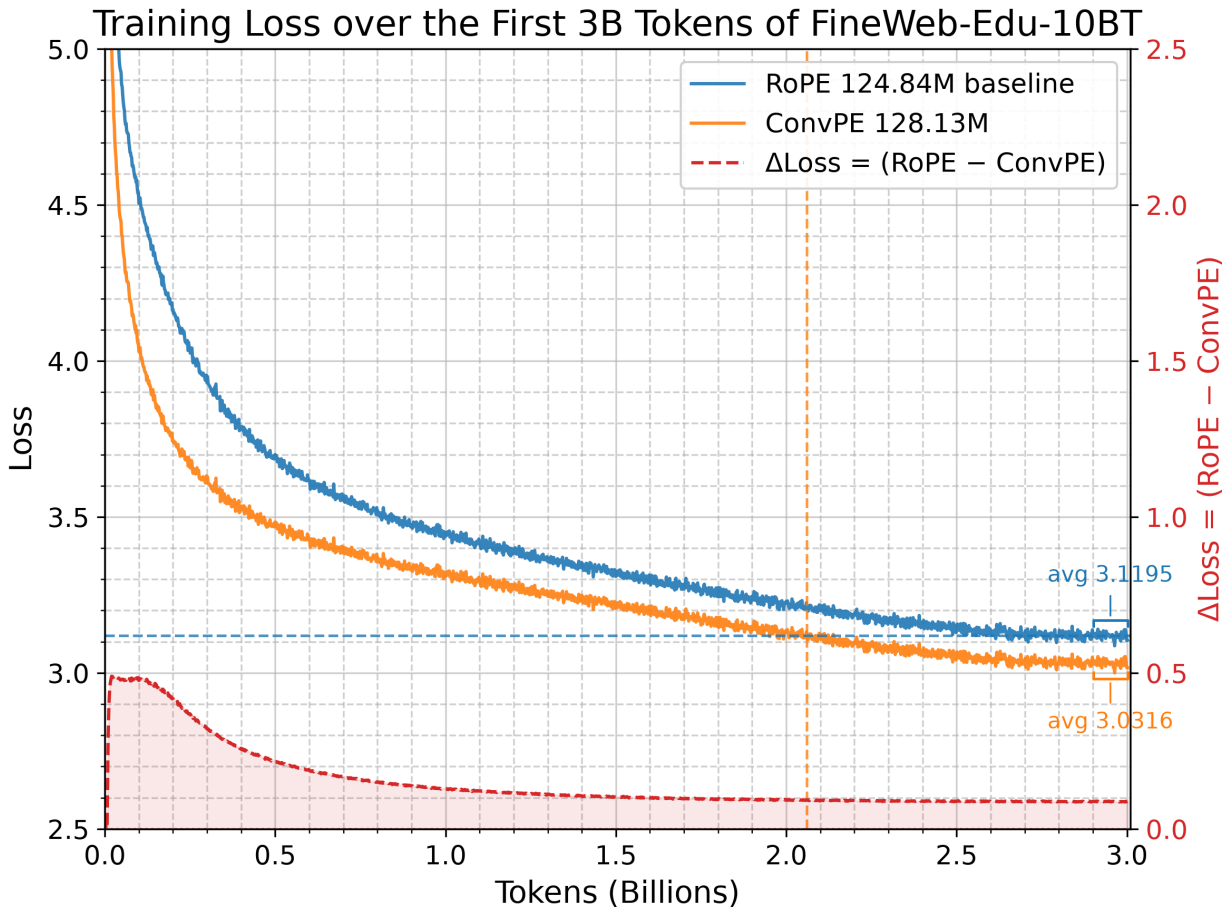


Figure 4: Pretraining curve on first 3B tokens of FineWeb-Edu-10BT

Figure 4 shows that ConvPE converges faster and achieves a lower final loss compared to the RoPE baseline. We implement high-performance Flash MQA kernels in TileLang and benchmark two configurations: `base RoPE` with 124.84M parameters and `base w128d16` with 128.13M parameters (+2.64%), using a batch size of 4 and a sequence length of 1024. While the forward-pass time increases by 2.70%, slightly exceeding the parameter increase, and the overall wall-clock time per step increases by approximately 7%, the final training loss drops from 3.1195 to 3.0316 (−2.82%). According to scaling laws[20,21], such a substantial loss reduction would typically require a model with $\sim 50\%$ more parameters. More importantly, despite the 7% per-step overhead, the number of training tokens required to match the RoPE baseline's final loss decreases from 3B to 2.06B, a 31.3% reduction. This translates to a net reduction in total wall-clock training time of approximately 26.5%.

4.3 2D Position Embedding

To produce 2D positional embeddings, we use 2D convolutional layers that take a 2D window input and produce a 2D window output. We use ViT-Tiny[22] models with Multi-Head Attention (MHA) and the configuration $N_{\text{layers}} = 12$, $D_{\text{model}} = 192$, $N_{\text{heads}} = 3$, and $D_{\text{head}} = 64$. We train the models on CIFAR-10 and CIFAR-100[23] for 100 epochs with patch shape 4×4.

Table 1 shows that ConvPE transfers effectively to vision transformers and outperforms the RoPE baseline. The influence of hyperparameters requires further study on larger models and datasets.

model	params	CIFAR-10	CIFAR-100
RoPE baseline	5.346M	0.910	0.627
ConvPE2D `w3d4`	5.359M	0.931	**0.695**
ConvPE2D `w3d8`	5.372M	**0.933**	0.683
ConvPE2D `w5d4`	5.402M	0.929	0.682
ConvNet	4.335M	**0.945**	**0.737**

Table 1: Validation accuracy on CIFAR-10 and CIFAR-100

5 Conclusions and Future Work

ConvPE delivers substantial performance gains at a minor parameter and computational cost, providing a powerful alternative for positional embeddings. We adopt $H_b = 1$ without any parameter-saving technique as the default configuration, which achieves the best balance among performance, parameter efficiency, and computational cost. ConvPE is compatible with RoPE, various attention masks, and KV-cache concatenation, and it naturally supports multidimensional extensions.

Future work can explore the following directions:

- **Kernel Optimization.** Further optimize the backward kernel and develop additional kernels for a wider range of hardware.
- **Scaling behavior.** Investigate ConvPE with larger Transformers and datasets, and identify configuration patterns at scale.
- **Long-context modeling.** Evaluate ConvPE in long-context settings, where positional generalization is particularly important.
- **Architectural integration.** Combine ConvPE with Sliding Window Attention (SWA)[24], Compressed Sparse Attention (CSA)[13], and other Transformer variants.
- **Multidimensional modalities.** Extend ConvPE to high-dimensional modalities such as vision, speech, and multimodal data.

References

[1] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

[2] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of NAACL-HLT*, pages 4171–4186, 2019.

[3] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. *OpenAI Blog*, 1(8):9, 2019.

[4] Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding. *arXiv preprint arXiv:2104.09864*, 2021.

[5] Lei Wang, Yu Cheng, Yining Shi, Zhengju Tang, Zhiwen Mo, Wenhao Xie, Lingxiao Ma, Yuqing Xia, Jilong Xue, Fan Yang, and Zhi Yang. Tilelang: A composable tiled programming model for AI systems. *CoRR*, abs/2504.17577, 2025. doi:10.48550/ARXIV.2504.17577. URL `https://doi.org/10.48550/arXiv.2504.17577`.

[6] Bowen Peng, Jeffrey Quesnelle, Honglu Fan, and Enrico Shippole. YaRN: Efficient context window extension of large language models. In *The Twelfth International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=wHBfxhZu1u.

[7] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research*, 21(140):1–67, 2020.

[8] Ofir Press, Noah A. Smith, and Mike Lewis. Train short, test long: Attention with linear biases enables input length extrapolation. *arXiv preprint arXiv:2108.12409*, 2021.

[9] Tyler A. Chang, Yifan Xu, Weijian Xu, and Zhuowen Tu. Convolutions and self-attention: Re-interpreting relative positions in pre-trained language models, 2021. URL https://arxiv.org/abs/2106.05505.

[10] Amirhossein Kazemnejad, Inkit Padhi, Karthikeyan Natesan Ramamurthy, Payel Das, and Siva Reddy. The impact of positional encoding on length generalization in transformers. *arXiv preprint arXiv:2305.19466*, 2023.

[11] Kimi Team. Kimi k3: Open frontier intelligence. *arXiv preprint arXiv:2607.24653*, 2026. URL https://arxiv.org/abs/2607.24653.

[12] Noam Shazeer. Fast transformer decoding: One write-head is all you need. *arXiv preprint arXiv:1911.02150*, 2019.

[13] DeepSeek-AI. Deepseek-v4: Towards highly efficient million-token context intelligence. *arXiv preprint arXiv:2606.19348*, 2026. URL https://arxiv.org/abs/2606.19348.

[14] Noam Shazeer. Glu variants improve transformer. *arXiv preprint arXiv:2002.05202*, 2020.

[15] Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. Pointer sentinel mixture models. *arXiv preprint arXiv:1609.07843*, 2016.

[16] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Remi Louf, Morgan Funtowicz, et al. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 38–45, 2020.

[17] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019.

[18] Keller Jordan. Muon: An optimizer for hidden layers in neural networks. https://kellerjordan.github.io/posts/muon/, 2024.

[19] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In Yoshua Bengio and Yann LeCun, editors, *3rd International Conference on Learning Representations, ICLR 2015, San Diego, CA, USA, May 7-9, 2015, Conference Track Proceedings*, 2015. URL http://arxiv.org/abs/1412.6980.

[20] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020. doi:10.48550/arXiv.2001.08361.

[21] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, et al. Training compute-optimal large language models. *arXiv preprint arXiv:2203.15556*, 2022. doi:10.48550/arXiv.2203.15556.

[22] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, et al. An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations*, 2021.

[23] Alex Krizhevsky. Learning multiple layers of features from tiny images. Technical report, University of Toronto, 2009.

[24] Iz Beltagy, Matthew E. Peters, and Arman Cohan. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*, 2020.